

PayLane: A Reserve-Gated Stablecoin Treasury on Solana

Dipankar Sarkar

2026

Abstract

Fiat-backed stablecoins are the settlement layer of on-chain payments, yet their solvency is enforced off-chain: an issuer holds reserves at a bank and promises that circulating supply never exceeds them. When that promise breaks, holders learn only after the fact. PayLane moves the promise into the program. It is a Solana treasury, written in Anchor, whose `mint` and `settle` instructions run a reserve gate that provably cannot push circulating supply past the reserves attested on-chain. The gate is a small, total function over unsigned 64-bit integers with no floating point and no administrative bypass; a mint of one unit beyond the attested reserve is rejected deterministically and leaves state untouched. Circulating supply is never a number the program stores and hopes is right: it is read from the bound `SPL Mint.supply` on every instruction, and redemption burns the holder's tokens, so the ledger cannot drift from the tokens that actually exist. An attestation that reports less backing than is circulating is recorded rather than refused, and pauses minting until solvency returns. We port the ProofBackedUSD reserve-gate idea from an EVM contract to Anchor, factor the invariant into a pure-Rust module that is unit-tested without the Solana toolchain, and exercise the instructions themselves against the real SPL Token program under `solana-program-test`. The result is a payment rail whose solvency is a property of the code, checkable by anyone, rather than a claim of the operator.

1 Problem

A fiat-backed stablecoin issuer maintains two numbers: the reserves it holds and the supply it has minted. The system is solvent exactly when supply does not exceed reserves. On every major stablecoin today this inequality is maintained by policy and audited periodically, not enforced at the point of minting. The mint authority is a key; whoever holds it can, in principle, mint beyond reserves, whether by error, by compromise, or by choice. Redemption at par then depends on trust that this has not happened between audits.

This is the wrong place for the check. Minting is

an on-chain action; the reserve figure can be attested on-chain; and the comparison between them is a single integer inequality. There is no technical reason the chain cannot refuse an over-mint the way it refuses a double-spend. The obstacle is that the check has historically lived in the issuer's back office rather than in the program that controls the mint. PayLane relocates it.

2 Related Work

Collateralized stablecoins such as MakerDAO's DAI enforce over-collateralization on-chain through per-vault liquidation, a different model in which each borrower's position is independently backed by volatile crypto collateral [1]. PayLane instead targets the fiat-backed model, where a single treasury attests a scalar reserve figure and the invariant is a global supply ceiling. Transparency efforts for fiat-backed issuers, such as Circle's published reserve attestations [2], make the reserve figure visible but do not bind minting to it; the mint remains a trusted key. Proof-of-reserves work in the exchange setting [3] likewise proves a snapshot without gating future issuance against it. On Solana specifically, the SPL Token program provides a mint authority abstraction [4] but delegates all policy to that authority; Anchor [5] is the framework we use to wrap the mint authority in a program-derived treasury that interposes the reserve gate. PayLane's contribution is to make the attested reserve a hard, program-enforced ceiling on issuance rather than a disclosed number.

3 Design

The treasury is a single program-derived account holding four fields relevant to solvency: the `authority` that may mint and settle, the `attestor` that may update reserves, the bound SPL mint, one unsigned 64-bit `attested_reserve` denominated in the token's smallest unit, and a pause flag. Circulating supply is deliberately *not* stored: it is projected from the bound mint's `supply` field at instruction time. The solvency invariant

is

`circulating ≤ attested_reserve`

and every state transition preserves it.

The gate. Minting an amount a is allowed only when `circulating + a ≤ attested_reserve`, computed with a checked addition so that a would-be 2^{64} overflow is rejected rather than wrapped. Settlement lowers circulating supply and cannot break the invariant, but is guarded against underflow. A new attestation is always recorded, including one that drops reserves below what is already circulating: that figure is the truth about a shortfall, and refusing it would leave a stale, higher number on the books while the issuer kept minting against reserves that no longer exist. Such an attestation instead pauses the treasury, and every mint is refused until an attestation covers supply again or holders redeem back under the line. The whole gate is a total function: every input maps to either a new valid state or a typed rejection, and a rejection is a no-op on state.

One source of truth. The gate lives in a pure-Rust module, `reserve.rs`, that has no dependency on the Anchor runtime. The on-chain program imports it verbatim and calls it inside each instruction handler; a sibling crate imports the same file and unit-tests it with plain `cargo test`. The security-critical logic that runs on-chain is therefore byte-identical to the logic under test, and the tests need neither a validator nor the Solana BPF toolchain to run.

Separated powers. The authority and the attester are distinct keys. The party that mints cannot raise its own reserve ceiling, and the party that attests reserves cannot mint. This mirrors the operational separation between a treasury desk and a proof-of-reserve oracle.

Agent and adapter seam. An off-chain agent, built on a small reusable agent core, drives the treasury. It reads treasury state, proposes mints and settlements, and runs the *same* reserve gate off-chain before submitting a transaction, so it never broadcasts an instruction the program would reject. Each action is journaled with a guardrail decision, giving an auditable activity log. The agent talks to the chain through an adapter seam whose rule is that submission comes first: nothing is recorded locally until a transaction confirms, and a failed submission leaves the mirror untouched. This build ships no live client, because nothing is deployed; when the seam is configured for a cluster and no client is supplied it refuses to construct rather than run the simulation and call it live. The deterministic offline mirror is labelled as such, down to the event digests, so the demo is keyless without claiming to be on-chain.

4 Hero

The hero action is an attempted over-mint. With one million dollars attested and eight hundred thousand circulating, the agent proposes minting three hundred thousand more. That would put circulating supply at 1,100,000 against 1,000,000 of backing. The gate computes $800,000 + 300,000 = 1,100,000 > 1,000,000$ and returns `ExceedsReserves`. No tokens are minted, no transaction is produced, and the treasury state is exactly what it was before. The operator then settles one hundred thousand out of circulation, which frees headroom, and the identical three-hundred-thousand mint now fits under the ceiling and is allowed. The point is visceral: solvency is not something the operator asserts between audits, it is something the program refuses to violate on every single mint.

5 Evaluation

We evaluate the gate directly, since it is the entire security claim, and then the program that wraps it. Nineteen pure-Rust unit tests exercise the invariant: mints within reserves, a mint exactly to the ceiling, a mint one unit past it, that a rejected mint does not mutate state, zero-amount rejection, overflow near 2^{64} , settlement lowering supply and freeing headroom, settlement underflow, an attestation below circulating supply being recorded and pausing the treasury, that no mint is possible while paused, that a covering attestation or a large enough redemption unpauses while a partial one does not, the backing ratio reporting the shortfall honestly and saturating instead of returning a sentinel, and a multi-step operator sequence that preserves the invariant at every step. They run in the Anchor crate and, byte for byte, in a sibling crate that needs no Solana toolchain.

Thirteen further tests drive the actual instructions against the real SPL Token program inside `solana-program-test`, an in-process bank. These assert on the SPL `Mint` account rather than on anything PayLane wrote about itself: that `initialize` moves mint authority to the treasury PDA and that a stranger cannot claim another issuer's mint, that a successful mint moves real supply and credits the holder while a rejected one moves nothing, that redemption burns the holder's tokens so settlement cannot manufacture unbacked headroom, that a recorded reserve loss pauses minting on-chain, and that the attester and the minting authority cannot act for each other. A stubbed mint helper fails every one of them.

A Python suite of forty-four tests re-checks the same invariant in the off-chain port case for case, pins the Anchor wire format (instruction discriminators and the `Treasury` account layout), proves the adapter seam

commits nothing when a submission fails, and exercises the CLI workflow and the guardrailed audit trail end-to-end.

[5] Coral / Anchor contributors. *The Anchor Framework*. <https://www.anchor-lang.com/>

Suite	Result
Rust reserve invariant (<code>cargo test</code>)	39 / 39
Rust program vs. SPL Token (<code>cargo test</code>)	13 / 13
Python agent + seam (<code>pytest</code>)	44 / 44

Table 1: Test results. The Rust suite proves the on-chain gate; the Python suite proves the off-chain agent agrees with it.

6 Limitations

The reserve gate is the trust-minimized core, but it does not by itself make the whole system trustless. The attested reserve is only as good as the attester: a dishonest or compromised attester can post a reserve figure not backed by real custody, and the on-chain gate will honor it. PayLane narrows the trusted surface from “trust that supply never exceeds reserves” to “trust that the attested reserve reflects custody,” which is a smaller and more auditable claim, but it does not eliminate it; a production deployment would pair the gate with a proof-of-reserve oracle or a multi-party attester. Second, and most importantly for a reader judging what has been demonstrated: the program has *not* been deployed. Compiling it for deployment needs the Solana toolchain (`anchor build / cargo build-sbf`), which is not installed in this environment, so there is no devnet program id and no explorer-verifiable transaction. What is demonstrated is the program’s behaviour under the real SPL Token program in an in-process bank, which is the same code path minus the SBF compilation and the network. Only devnet deployment is in scope; mainnet is not. Third, the treasury is bound to a single mint and a single attester key; multi-currency treasuries and threshold attestation are future work.

References

- [1] MakerDAO. *The Maker Protocol White Paper*. 2020. <https://makerdao.com/en/whitepaper>
- [2] Circle. *USDC Transparency and Reserves*. <https://www.circle.com/en/transparency>
- [3] G. Maxwell et al. *Provisions: Privacy-preserving proofs of solvency for Bitcoin exchanges*. ACM CCS, 2015. <https://eprint.iacr.org/2015/1008>
- [4] Solana Labs. *SPL Token Program*. <https://spl.solana.com/token>